

Programação Orientada a Objetos com Java

Guia Completo para Iniciantes

O que é Programação Orientada a Objetos?

A Programação Orientada a Objetos (POO) é uma forma de organizar o código que imita o mundo real. Em vez de escrever instruções soltas, você cria objetos que representam coisas do mundo — um produto, um aluno, uma conta bancária.

Analogia do Mundo Real

Pense em um carro:

- Todo carro tem atributos: cor, marca, velocidade
- Todo carro tem comportamentos: acelerar, frear, ligar
- Cada carro é um objeto diferente, mas todos seguem o mesmo molde (classe)

Em POO: a Classe é o molde. O Objeto é o carro concreto criado a partir dele!

Os 4 Pilares da POO

Pilar	O que significa	Analogia
Encapsulamento	Esconder detalhes internos	Caixa preta do avião
Herança	Filho herda do pai	Filho herda traços dos pais
Polimorfismo	Mesmo nome, comportamentos diferentes	Botão ligar no TV e no carro
Abstração	Focar no essencial	Dirigir sem saber o motor por dentro

Classes e Objetos

3.1 Classe e Objeto

Uma Classe é um molde. Um Objeto é uma instância concreta criada a partir desse molde. Pense em uma forma de biscoito (classe) e os biscoitos que você faz com ela (objetos).

```
// Definindo a classe (o molde)
public class Carro {
    String marca;
    String cor;
    int ano;

    public void ligar() {
        System.out.println(marca + " ligado!");
    }
}
```

```

    }
}

// Criando objetos (instâncias)
Carro meuCarro = new Carro();
meuCarro.marca = "Toyota";
meuCarro.cor = "Prata";
meuCarro.ligar(); // Toyota ligado!

```

3.2 Atributos e Métodos

Definições Rápidas

- Atributos — características/variáveis da classe (cor, nome, preço...)
- Métodos — comportamentos/ações da classe (calcular, exibir, atualizar...)
- this — palavra reservada que se refere ao próprio objeto atual

```

public class Produto {
    // ATRIBUTOS
    String nome;
    double preco;
    int quantidade;

    // MÉTODOS
    public void exibir() {
        System.out.println("Produto: " + nome + " | R$" + preco);
    }

    public double calcularTotal() {
        return preco * quantidade;
    }
}

```

3.3 Construtores

O construtor é um método especial executado automaticamente quando criamos um objeto com new. Ele inicializa os atributos do objeto.

```

public class Produto {
    String nome;
    double preco;

    // CONSTRUTOR - mesmo nome da classe, sem tipo de retorno
    public Produto(String nome, double preco) {
        this.nome = nome; // this.nome = atributo da classe
        this.preco = preco; // nome (sem this) = parâmetro
    }

    public void exibir() {
        System.out.println(nome + " - R$ " + preco);
    }
}

```

```

    }

    public static void main(String[] args) {
        Produto p1 = new Produto("Caneta", 2.50);
        Produto p2 = new Produto("Caderno", 15.90);
        p1.exibir();
        p2.exibir();
    }
}

```

Lista de Exercícios

Crie uma classe Java completa para cada exercício:

Exercício 1: Classe Aluno

Crie uma classe Aluno com atributos nome, matricula e nota. Adicione construtor e método exibir().

```

public class Aluno {
    String nome; int matricula; double nota;

    public Aluno(String nome, int matricula, double nota) {
        this.nome = nome; this.matricula = matricula; this.nota = nota;
    }

    public void exibir() {
        System.out.println("Aluno: " + nome + " | Mat: " + matricula);
        System.out.println("Nota: " + nota + (nota >= 6 ? " - APROVADO" : " - REPROVADO"));
    }
}

```

Exercício 2: Classe Livro

Crie uma classe Livro com titulo, autor e preco. Inclua construtor e método exibir().

```

public class Livro {
    String titulo, autor; double preco;

    public Livro(String titulo, String autor, double preco) {
        this.titulo = titulo; this.autor = autor; this.preco = preco;
    }

    public void exibir() {
        System.out.printf("%s por %s - R$%.2f%n", titulo, autor, preco);
    }
}

```

Exercício 3: Classe ContaBancaria

Crie ContaBancaria com numero, titular e saldo. Inclua depositar(), sacar() e exibirSaldo().

```

public class ContaBancaria {
    int numero; String titular; double saldo;

    public ContaBancaria(int numero, String titular) {
        this.numero = numero; this.titular = titular; this.saldo = 0;
    }

    public void depositar(double valor) { saldo += valor; }
    public void sacar(double valor) {
        if (valor <= saldo) saldo -= valor;
        else System.out.println("Saldo insuficiente!");
    }

    public void exibirSaldo() {
        System.out.printf("Conta %d | %s | R$%.2f%n", numero, titular, saldo);
    }
}

```

Exercício 4: Classe Produto

Crie a classe Produto conforme o exemplo da teoria e instancie 3 produtos diferentes no main().

```

public class Produto {
    String nome; double preco; int quantidade;

    public Produto(String nome, double preco, int quantidade) {
        this.nome = nome; this.preco = preco; this.quantidade = quantidade;
    }

    public void exibir() {
        System.out.printf("%s | R$%.2f | Qtd: %d%n", nome, preco, quantidade);
    }

    public static void main(String[] args) {
        new Produto("Caneta", 2.50, 100).exibir();
        new Produto("Caderno", 15.90, 30).exibir();
        new Produto("Borracha", 1.20, 200).exibir();
    }
}

```

Mini Sistema de Estoque

Crie um sistema com menu que use a classe Produto para cadastrar, listar e calcular o total em estoque.

- Use um array Produto[] com capacidade para 10 itens
- Menu: 1-Cadastrar, 2-Listar, 3-Total em estoque, 4-Sair
- Total = soma de (preco * quantidade) de todos os produtos

Encapsulamento e Herança

4.1 Encapsulamento

Encapsulamento significa proteger os dados de uma classe, controlando quem pode acessar e modificar seus atributos. Usamos modificadores de acesso: `private`, `public` e `protected`.

Modificadores de Acesso

- `public` — qualquer classe pode acessar
- `private` — só a própria classe pode acessar
- `protected` — a própria classe e as subclasses podem acessar

4.2 Getters e Setters

Quando tornamos atributos `private`, criamos métodos `get` (para ler) e `set` (para modificar) que funcionam como porteiros controlados.

```
public class ContaBancaria {
    private String titular; // private = ninguém acessa diretamente
    private double saldo;

    public ContaBancaria(String titular) {
        this.titular = titular; this.saldo = 0;
    }

    // GETTER - lê o valor
    public String getTitular() { return titular; }
    public double getSaldo() { return saldo; }

    // Métodos com validação
    public void depositar(double valor) {
        if (valor > 0) saldo += valor;
        else System.out.println("Valor invalido!");
    }

    public void sacar(double valor) {
        if (valor > 0 && valor <= saldo) saldo -= valor;
        else System.out.println("Operacao nao permitida!");
    }
}
```

4.3 Herança

Herança permite que uma classe filha herde atributos e métodos de uma classe pai. Usamos a palavra `extends`. A classe filha pode usar tudo que o pai tem e adicionar o que quiser.

```
// CLASSE PAI
public class Pessoa {
```

```

protected String nome;
protected int idade;

public Pessoa(String nome, int idade) {
    this.nome = nome; this.idade = idade;
}

public void apresentar() {
    System.out.println("Ola, sou " + nome + ", tenho " + idade + " anos.");
}
}

// CLASSE FILHA — herda de Pessoa
public class Aluno extends Pessoa {
    private String matricula;

    public Aluno(String nome, int idade, String matricula) {
        super(nome, idade); // chama construtor do pai
        this.matricula = matricula;
    }

    public void exibirMatricula() {
        System.out.println("Matricula: " + matricula);
    }
}

// USO
Aluno a = new Aluno("Maria", 20, "2024001");
a.apresentar(); // herdado de Pessoa
a.exibirMatricula(); // proprio de Aluno

```

A palavra super

- `super(args)` — chama o construtor da classe pai
- `super.metodo()` — chama um método da classe pai

Sempre que a classe filha tem construtor, precisa chamar `super()` como primeiro comando!

Lista de Exercícios

Pratique encapsulamento e herança:

Exercício 1: Hierarquia Pessoa → Aluno e Professor

Crie Pessoa com nome e idade. Aluno herda e adiciona matrícula. Professor herda e adiciona disciplina e salário.

```

public class Pessoa {
    protected String nome; protected int idade;
    public Pessoa(String n, int i) { nome = n; idade = i; }
    public void apresentar() {
        System.out.println("Nome: " + nome + " | Idade: " + idade);
    }
}

```

```

public class Aluno extends Pessoa {
    private String matricula;
    public Aluno(String n, int i, String m) { super(n, i); matricula = m; }
    public void exibir() { apresentar(); System.out.println("Mat: " + matricula); }
}

public class Professor extends Pessoa {
    private String disciplina; private double salario;
    public Professor(String n, int i, String d, double s) {
        super(n, i); disciplina = d; salario = s;
    }
    public void exibir() {
        apresentar();
        System.out.printf("Disc: %s | Sal: R$%.2f%n", disciplina, salario);
    }
}

```

Exercício 2: Hierarquia Veiculo → Carro e Moto

Crie Veiculo com marca, ano e velocidadeMax. Carro herda e tem portas. Moto herda e tem cilindradas.

```

public class Veiculo {
    protected String marca; protected int ano; protected double velMax;
    public Veiculo(String m, int a, double v) { marca=m; ano=a; velMax=v; }
    public void exibir() { System.out.printf("%s (%d) - %.0f km/h%n", marca, ano, velMax); }
}

public class Carro extends Veiculo {
    private int portas;
    public Carro(String m, int a, double v, int p) { super(m,a,v); portas=p; }
    public void exibir() { super.exibir(); System.out.println("Portas: " + portas); }
}

public class Moto extends Veiculo {
    private int cilindradas;
    public Moto(String m, int a, double v, int c) { super(m,a,v); cilindradas=c; }
    public void exibir() { super.exibir(); System.out.println("Cil: " + cilindradas + "cc"); }
}

```

Sistema Acadêmico Simples

Crie um sistema usando as classes Pessoa, Aluno e Professor com encapsulamento correto.

- Menu: 1-Cadastrar Aluno, 2-Cadastrar Professor, 3-Listar Alunos, 4-Listar Professores
- Use arrays separados para alunos e professores
- Todos os atributos devem ser private com getters

Polimorfismo e Abstração

5.1 Polimorfismo

Polimorfismo significa muitas formas. É quando objetos de tipos diferentes respondem ao mesmo método de formas diferentes. Um cachorro late, um gato mia, um pássaro canta — mas todos respondem ao falar()!

```
public class Animal {
    public void falar() { System.out.println("..."); }
}

public class Cachorro extends Animal {
    @Override
    public void falar() { System.out.println("Au au!"); }
}

public class Gato extends Animal {
    @Override
    public void falar() { System.out.println("Miau!"); }
}

// USO POLIMÓRFICO
Animal[] animais = {new Cachorro(), new Gato(), new Cachorro()};
for (Animal a : animais) {
    a.falar(); // cada um fala do seu jeito!
}
```

5.2 Classes Abstratas

Uma classe abstrata é um molde incompleto — define o que as subclasses devem implementar, mas não diz como. Você não pode criar objetos diretos de uma classe abstrata.

```
public abstract class Animal {
    protected String nome;
    public Animal(String nome) { this.nome = nome; }

    // Método abstrato: obriga as filhas a implementar
    public abstract void falar();

    // Método concreto: já implementado
    public void dormir() {
        System.out.println(nome + " esta dormindo...");
    }
}

public class Passaro extends Animal {
    public Passaro(String nome) { super(nome); }

    @Override
    public void falar() { System.out.println(nome + ": Piu piu!"); }
}
```

```
// ERRO: Animal a = new Animal(); // nao pode instanciar abstrato
Passaro p = new Passaro("Tweety");
p.falar(); // Tweety: Piu piu!
p.dormir(); // Tweety esta dormindo...
```

5.3 Interfaces

Uma interface é um contrato: define quais métodos uma classe deve ter, sem implementar nenhum. Uma classe pode implementar várias interfaces ao mesmo tempo.

```
// A interface define o CONTRATO
public interface Pagavel {
    void processar(double valor);
    String getDescricao();
}

// Pix implementa o contrato
public class Pix implements Pagavel {
    private String chave;
    public Pix(String chave) { this.chave = chave; }

    @Override
    public void processar(double valor) {
        System.out.printf("Pix de R$%.2f para %s\n", valor, chave);
    }
    @Override
    public String getDescricao() { return "PIX"; }
}

// Cartao implementa o mesmo contrato
public class Cartao implements Pagavel {
    private String numero; private int parcelas;
    public Cartao(String num, int parc) { numero = num; parcelas = parc; }

    @Override
    public void processar(double valor) {
        System.out.printf("Cartao: R$%.2f em %dx\n", valor, parcelas);
    }
    @Override
    public String getDescricao() { return "CARTAO"; }
}

// Boleto implementa o mesmo contrato
public class Boleto implements Pagavel {
    @Override
    public void processar(double valor) {
        System.out.printf("Boleto: R$%.2f (vence em 3 dias)\n", valor);
    }
    @Override
    public String getDescricao() { return "BOLETO"; }
}
```

Classe Abstrata vs Interface

- Classe Abstrata → use quando subclasses compartilham código em comum

- Interface → use quando classes diferentes precisam de um comportamento em comum
- Uma classe pode herdar apenas UMA classe abstrata
- Uma classe pode implementar VÁRIAS interfaces

Lista de Exercícios

Pratique polimorfismo e interfaces:

Exercício 1: Hierarquia Animal com Polimorfismo

Crie a classe abstrata Animal com método abstrato falar(). Implemente Cachorro, Gato e Passaro. Percorra um array de Animal e chame falar() em cada um.

```
public abstract class Animal {
    protected String nome;
    public Animal(String nome) { this.nome = nome; }
    public abstract void falar();
    public void apresentar() {
        System.out.print(nome + " diz: "); falar();
    }
}

public class Cachorro extends Animal {
    public Cachorro(String n) { super(n); }
    public void falar() { System.out.println("Au au!"); }
}

public class Gato extends Animal {
    public Gato(String n) { super(n); }
    public void falar() { System.out.println("Miau!"); }
}

public class Passaro extends Animal {
    public Passaro(String n) { super(n); }
    public void falar() { System.out.println("Piu piu!"); }
}

// No main:
Animal[] bichos = {new Cachorro("Rex"), new Gato("Mimi"), new Passaro("Tweety")};
for (Animal a : bichos) a.apresentar();
```

Exercício 2: Interface Pagavel — Sistema de Pagamentos

Crie a interface Pagavel com processar() e getDescricao(). Implemente Pix, Cartao e Boleto. Crie um método que aceita Pagavel e processa qualquer forma de pagamento.

```
public interface Pagavel {
    void processar(double valor);
    String getDescricao();
}
```

```
// Método polimórfico – aceita qualquer Pagavel
public static void realizarPagamento(Pagavel metodo, double valor) {
    System.out.println("=== VIA " + metodo.getDescricao() + " ===");
    metodo.processar(valor);
}

// No main:
realizarPagamento(new Pix("pix@email.com"), 150.00);
realizarPagamento(new Cartao("1234", 3), 299.90);
realizarPagamento(new Boleto(), 89.90);
```

DESAFIO FINAL — Sistema de Pagamentos Completo

O que você vai construir:

Um sistema que integra todos os conceitos das 3 semanas: classes, herança, encapsulamento e interfaces.

- Interface Pagavel com processar() e getDescricao()
- Classes Pix, Cartao e Boleto implementando Pagavel
- Classe Pedido com produto, valor e forma de pagamento
- Menu: 1-Novo pedido, 2-Listar pedidos, 3-Total vendido, 4-Sair

Código do Projeto:

```
public class Pedido {
    private String produto;
    private double valor;
    private Pagavel pagamento; // usa a interface!

    public Pedido(String produto, double valor, Pagavel pagamento) {
        this.produto = produto;
        this.valor = valor;
        this.pagamento = pagamento;
    }

    public void finalizar() {
        System.out.println("--- Pedido: " + produto + " ---");
        System.out.printf("Valor: R$%.2f\n", valor);
        pagamento.processar(valor); // polimorfismo!
    }

    public double getValor() { return valor; }
}

// No main – usando polimorfismo com interface:
Pedido[] pedidos = new Pedido[10];
int total = 0; double totalVendido = 0;

// Cadastrando pedidos com diferentes formas de pagamento:
pedidos[total++] = new Pedido("Notebook", 3500.00, new Pix("loja@pix"));
pedidos[total++] = new Pedido("Mouse", 89.90, new Cartao("4321", 2));
```

```
pedidos[total++] = new Pedido("Teclado", 199.00, new Boleto());

// Listando e calculando total:
for (int i = 0; i < total; i++) {
    pedidos[i].finalizar();
    totalVendido += pedidos[i].getValor();
}
System.out.printf("Total vendido: R$%.2f%n", totalVendido);
```

Resumo dos Conceitos — Fase 2

Conceito	Palavra-chave	Para que serve
Classe	<code>class</code>	Molde dos objetos
Objeto	<code>new</code>	Instância concreta da classe
Construtor	<code>NomeDaClasse()</code>	Inicializar atributos
Encapsulamento	<code>private + get/set</code>	Proteger dados da classe
Herança	<code>extends</code>	Filha herda atributos e métodos do pai
Super	<code>super()</code>	Chamar construtor/método do pai
Polimorfismo	<code>@Override</code>	Mesmo método, comportamentos diferentes
Classe Abstrata	<code>abstract</code>	Molde incompleto — define obrigações
Interface	<code>implements</code>	Contrato de comportamento

Agora você pensa como um desenvolvedor Java de verdade. Continue praticando!